



Politecnico
di Torino

Runtime Security Monitoring System basato su eBPF

Pipeline end-to-end e implementazione hook

Simone Romano · Giuseppe Insalaco

Riferimento Progettuale e Vincoli Architeturali



Tracee (Riferimento)

- Runtime security general purpose e multi-kernel.
- Ring buffer come canale principale.
- Riferimento per probe registry, pipeline e modello degli eventi

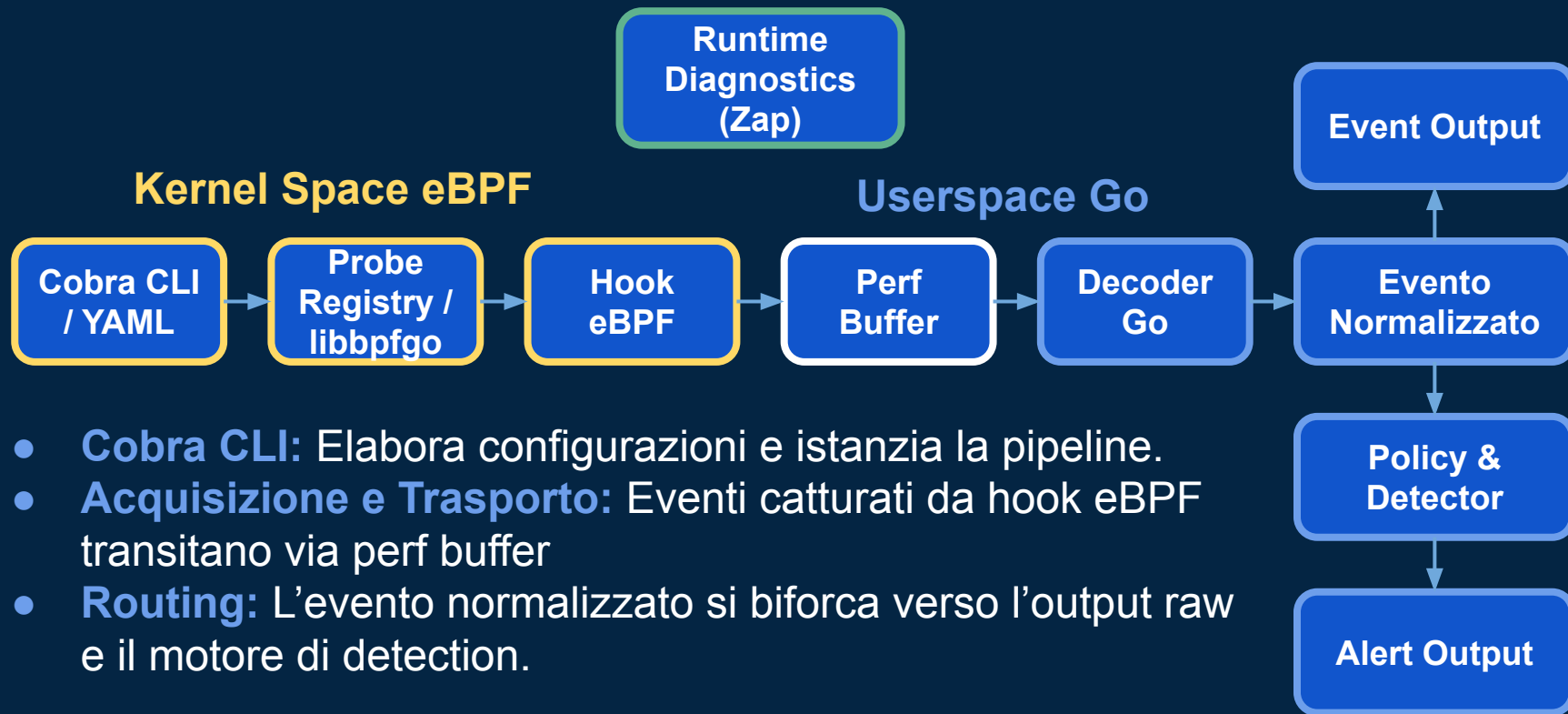


Nostro Approccio

- Target mirato e ottimizzato: Rocky Linux 8, kernel 4.18.
- Adattamento per vincoli del verifier (es. offset dimostrabili).
- Perf buffer come canale di trasporto operativo principale
- Policy layer essenziale

Non una replica di Tracee, ma un runtime più mirato e controllabile.

Architettura e Pipeline End-to-End



Contratto Binario e Normalizzazione degli Eventi

Kernel
[event_context_t + argnum + args buffer]

- **Contratto Binario:** Context comune, argomenti indicizzati e registry C/Go.
- **Mapping Univoco:** Associazione tra ID e eBPF, nome evento e schema di decodifica.
- **Arricchimento:** Normalizzazione di capability, flag, bitmask ed errno in userspace.



da formato binario

Registry Condiviso
C/Go

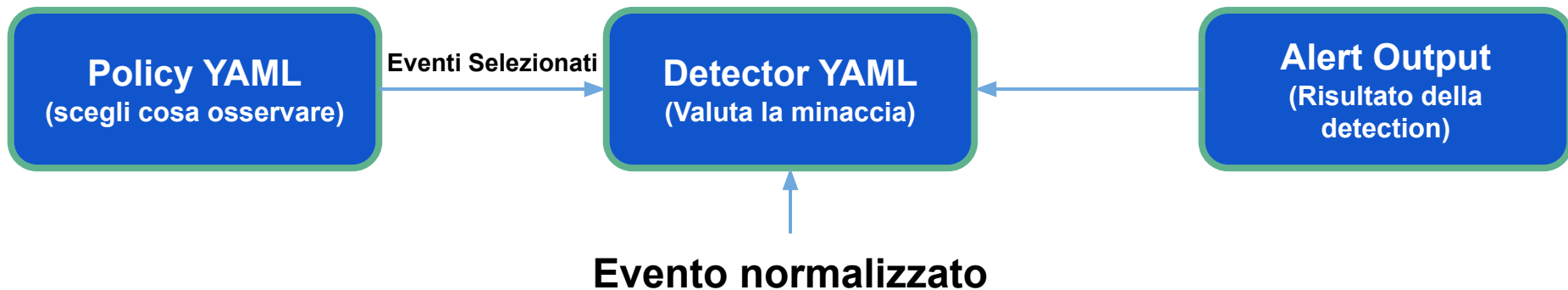


a tipizzati e indicizzati

Userspace

Evento:	security_file_open
Target:	/etc/shadow
Flag:	0_RDONLY

Layer di Policy e Detection (YAML)

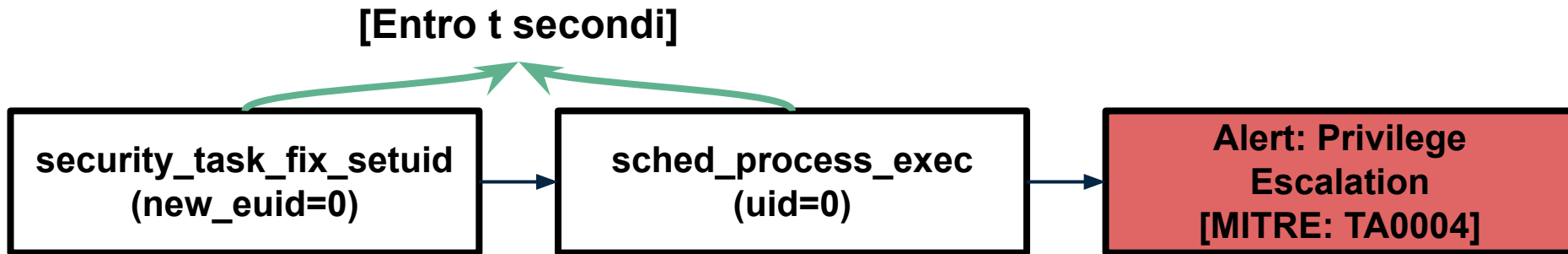


- **Policy:** Definisce lo scenario e seleziona gli eventi da osservare.
- **Detector:** Valuta condizioni dichiarative su uno o più eventi normalizzati.
- **Alert:** Rappresenta l'esito della detection, pronto per l'analista.

Policy e detector vengono validati prima del caricamento e dell'attach eBPF.

Point e Collective Detection (MITRE ATT&CK)

- **Point Anomaly:** Valutazione condizionale su singolo evento isolato.
- **Collective Anomaly:** Correlazione temporale di una breve sequenza di eventi.
- **Engine & Dedup:** Dedup temporale di 5 secondi applicato dall'engine per ridurre il rumore operativo
- **Metadata MITRE:** Tattiche e tecniche incluse nell'alert a scopo descrittivo.



Configurazione, Logging e Routing Output

- **Separazione Netta:** Zap gestisce il lifecycle; printer dedicati formattano eventi e alert in stream isolati (Table/JSON).
- **Modalità –alerts-only:** Nasconde l'output terminale degli eventi raw pur mantenendoli disponibili internamente per la pipeline di valutazione.
- **Preparazione al deployment Kubernetes:** Architettura pensata per container e Kubernetes grazie allo stream management indipendente.

Diagnostica

Runtime Lifecycle

Libreria Zap

Stderr
(Console/JSON)

Monitoraggio

Eventi eBPF
normalizzati

Event Printer

Stdout
(Table/JSON)

Sicurezza

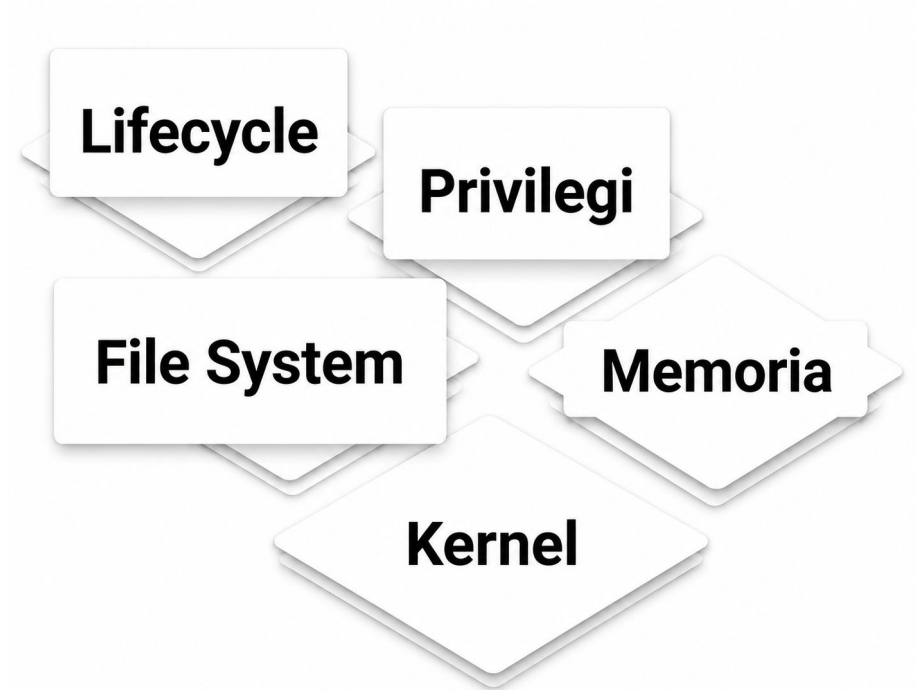
Alert dei detector

Alert Printer

Stdout
(Table/JSON)

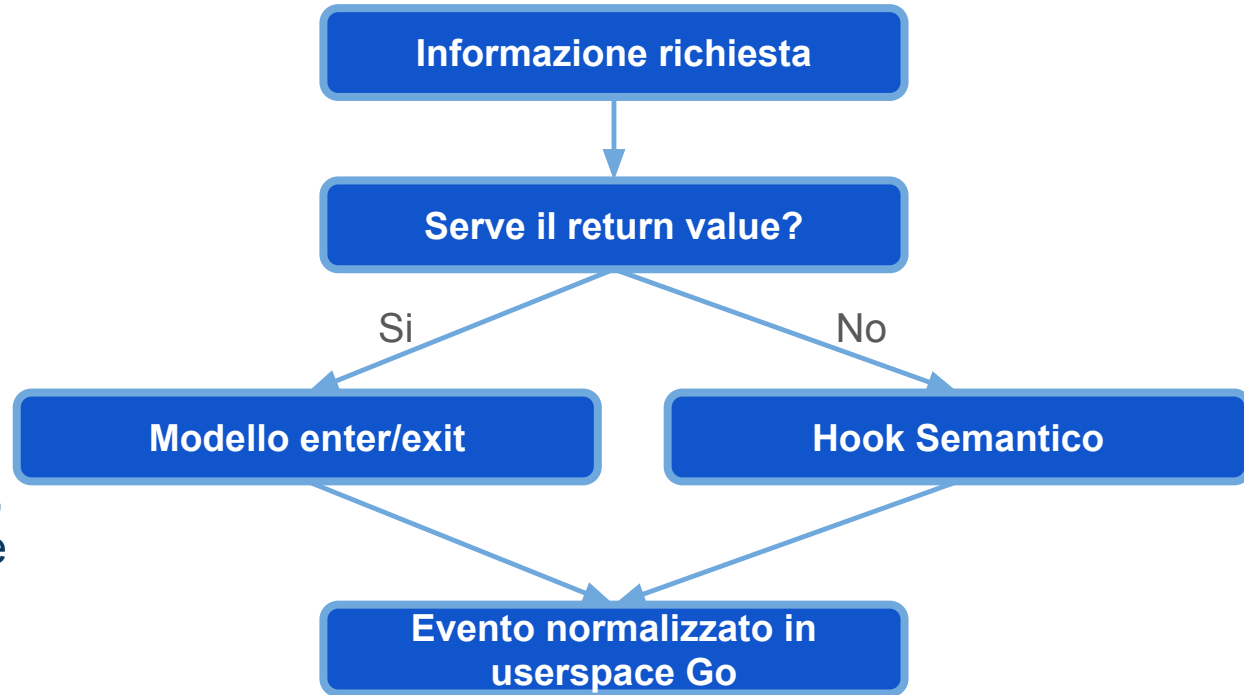
Focus: Superficie Process, Security e Kernel

- **Filosofia di Copertura:** Prediligere hook semantici (kprobe/security) rispetto a tracepoint generici rumorosi.
- **Controllo del Ritorno:** Modello enter/exit applicato selettivamente per intercettare gli esiti delle syscall.
- **Obiettivo:** Monitoraggio mirato delle minacce host-level, escludendo il layer di rete

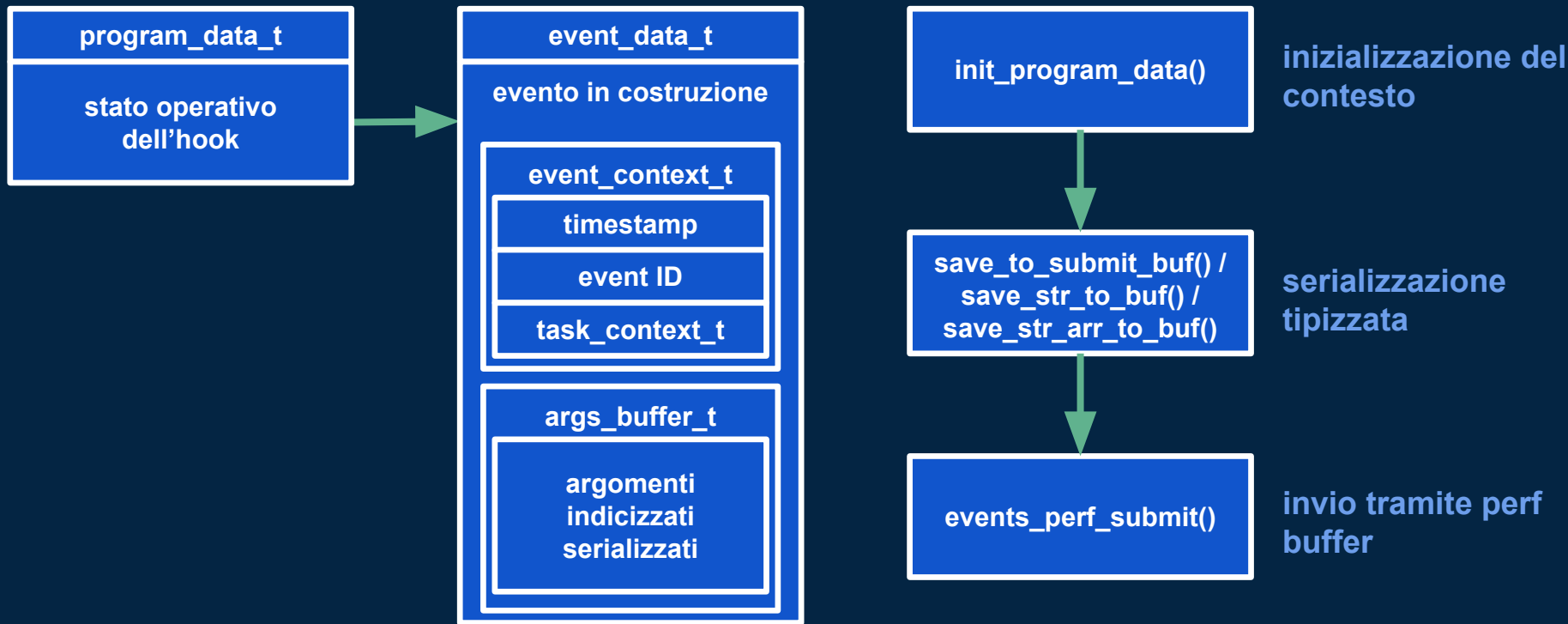


Strategia di Implementazione degli Hook

- **Hook Semantici:** Basato prevalentemente su kprobe e security hook. Sfrutta oggetti kernel già risolti e decodificati dal sistema operativo
- **Modello enter/exit:** Sfruttato quando serve conoscere il return value. L'entry salva gli argomenti, l'exit aggiunge l'esito finale
- Adattamento ai vincoli del verifier su **Rocky Linux 4.18**



Pattern comune di implementazione eBPF



Quando serve il return value, l'entry conserva gli argomenti e l'exit completa l'evento con l'esito della syscall.

Lifecycle: Creazione, esecuzione e Terminazione

fork / vfork / clone
Creazione Syscall:
Intercettano la richiesta iniziale.

task_rename
Transizione:
Traccia i cambi di nome (comm) in volo.

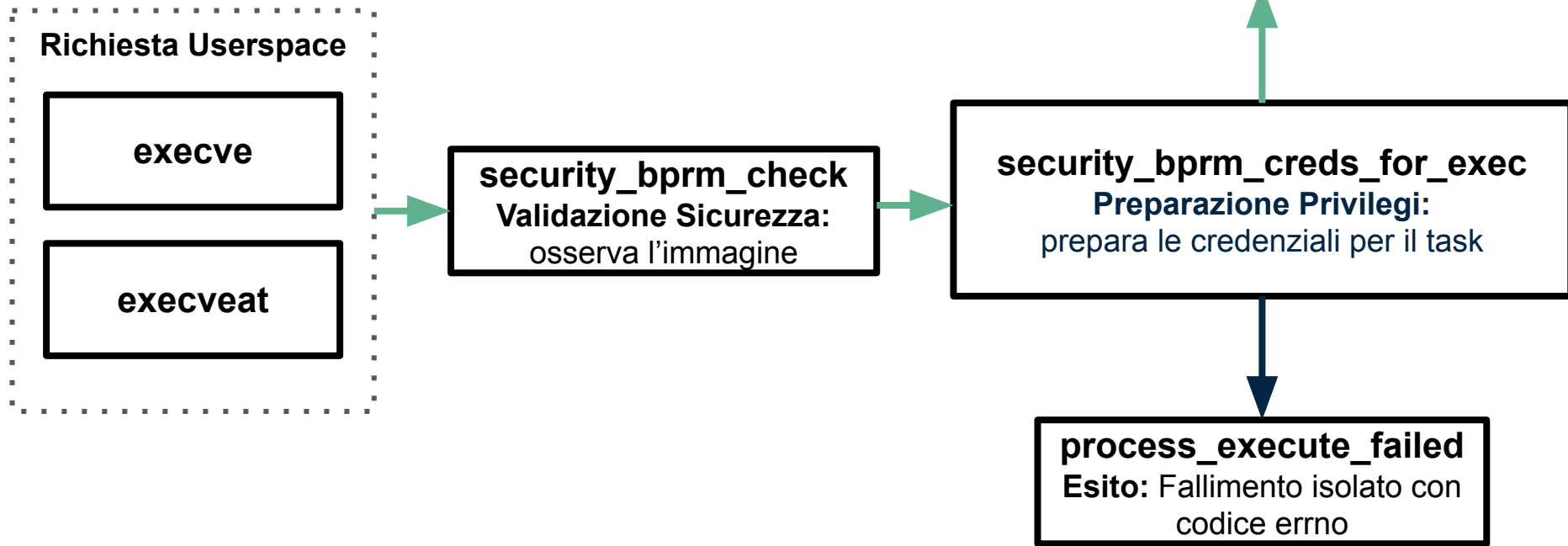
sched_process_exit
Terminazione:
Registra uscita ed exit code.

sched_process_fork
Formattazione Task:
Osserva la relazione parent-child nel kernel

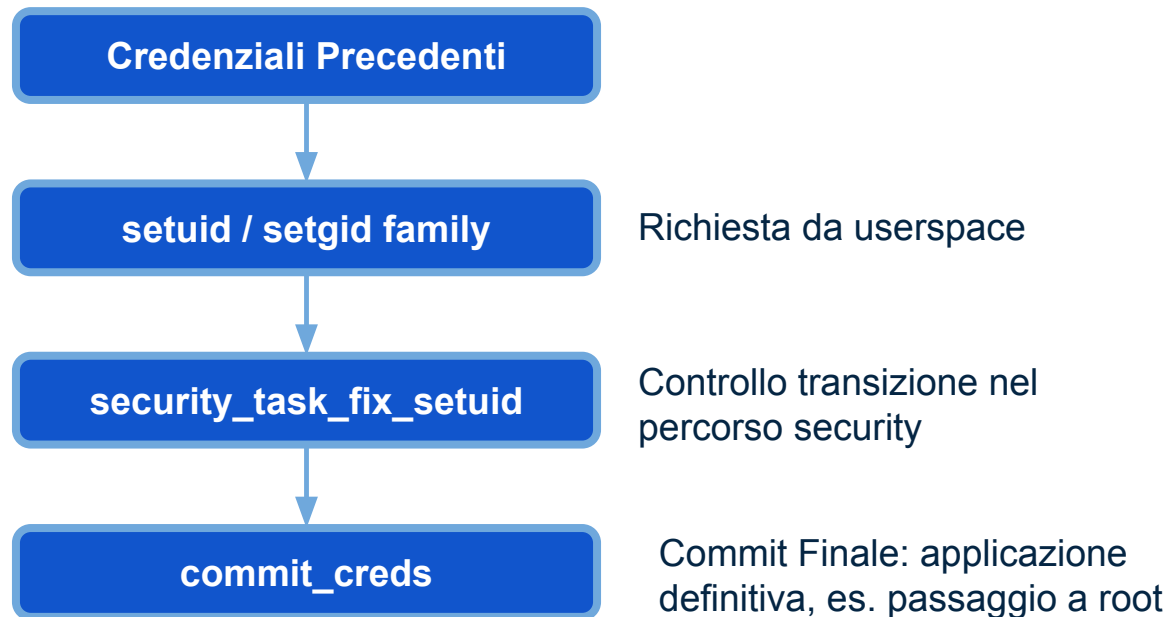
sched_process_exec
Immagine Esecutiva:
Conferma l'avvio riuscito



Exec Monitoring: Tentativi ed Esiti



Identità, Credenziali e Capability



Verifica e Manipolazione

- **cap_capable**: monitora l'uso di privilegi specifici (verifica capability).
- **ptrace & security_task_prctl**: osservano interazioni dirette e alterazioni dello stato dei processi.

Sicurezza File, Permessi e Filesystem

Accesso

- **open:** restituisce l'esito
- **security_file_open / permission:** espongono metadati stabili (path, inode, device, ctime).

Permessi

- **Famiglia chmod e chown:** intercettano modifiche rispettivamente a permessi e ownership, includendo il return value.

Inode

- **unlink, rename, symlink e mknod:** tracciano le alterazioni strutturali del filesystem.

Filesystem

- **security_sb_mount e umount:** osservano i mount point e i flag associati all'isolamento.

Memoria e Namespace

Gestione memoria

mmap

mprotect

pkey_mprotect

memfd_create

process_vm_readv / process_vm_writev

Monitoraggio transizioni a **memoria eseguibile**, visibilità su **fileless execution**, mapping anomali e accesso diretto alla memoria di processi terzi.

Transizioni Namespace

setns

unshare

switch_task_ns

Osservazione delle richieste di **isolamento** e verifica della loro effettiva applicazione nel kernel (es. **isolamento e runtime container**).

Kernel Hardening e Moduli

Caricamento Moduli:

module_load,
module_free,
do_init_module.

Superficie eBPF:

security_bpf,
security_bpf_map,
security_bpf_prog.

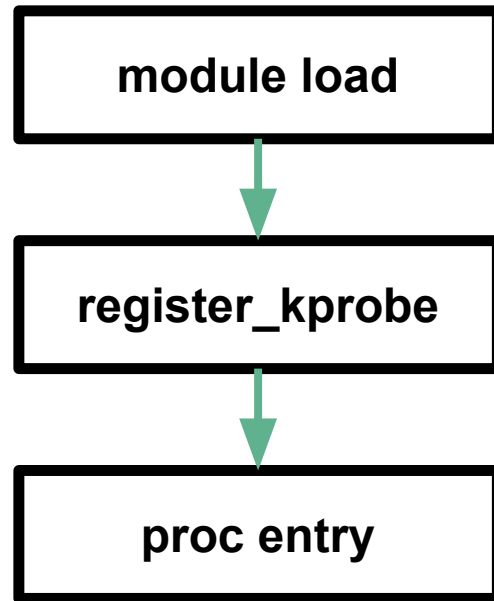
Strumentazione Dinamica:

register_kprobe,
kallsyms_lookup_name.

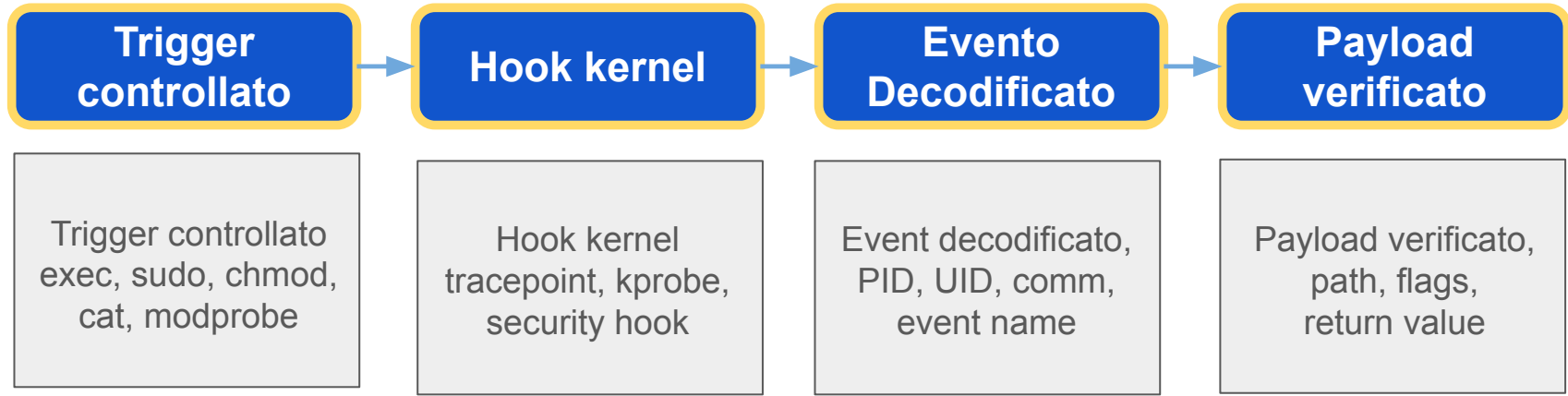
Interfacce Kernel:

proc_create,
call_usermodehelper,
security_kernel_read_file.

Esempio di scenario osservabile
tramite eventi distinti



Testing e Validazione su Rocky Linux 4.18



- **Obiettivo globale:** Target <5% di un core (CPU in fase di ottimizzazione)
- **CPU** (VM 2 core, Kernel 4.18): Media stabile al **5,9% - 7,7%**, con picco di startup al **16,92%**
- **Memoria:** Footprint contenuto a circa **45 MiB (RSS)**

Limiti Rilevati

- Rumore e overhead di alcuni hook.
- Esito syscall non disponibile per tutti gli eventi.
- Copertura avanzata e detector collective ancora da consolidare.

Step Tecnici Pianificati

- Filtri kernel-side e throttling degli alert.
- Estensione degli hook e del modello enter/exit.
- Benchmark automatici kernel/userspace sotto carico.

PLACEHOLDER PER PARTE DI GIUSEPPE